# Clone and Copy Discipline in `f1r3node-rust`

Where the copies are, which ones matter, and how to remove them without moving consensus

F1R3FLY.io Research Note

Prepared with Claude for Meredith

20 October 2026 • `dev` @ `e8008c3`

**Abstract**

This note audits the `dev` branch of `f1r3node-rust` for unnecessary `clone()`, `to_vec()`, `cloned()` and `to_owned()` calls, with the aim of finding the largest performance gains that leave consensus behaviour untouched. Production code (tests and scripts excluded) contains about 3,800 explicit clones in roughly 268,000 lines. The raw count is a poor guide: block hashes, state hashes and validator keys are already reference-counted `Bytes`, so most casper clones are pointer bumps. The expensive copies cluster in a small number of places where a value with deep structure — the protobuf `Par` tree, an `Env`, a block body, a deploy-chain index, a stream buffer — is copied because the code transliterates a Scala idiom (persistent, structurally shared collections) into Rust collections that copy deeply. Seven of these sites are *asymptotic* defects (quadratic in block size, binding count, COMM count or chain length). Microbenchmarks of the exact patterns show 17–270× slowdowns at realistic sizes, and 126× (5.6 s versus 44 ms) for reassembling a 64 MiB stream. The note ranks the findings into four tiers, separates the clones that are *semantically required* (random-number splitting, cost order, canonical sorting, hash bytes) from those that are not, and proposes a sequence of small pull requests, each with a determinism guard.

## Contents

**1 Scope and method** **3**

**2 Census** **3**

**3 What a clone costs here** **3**

**4 Findings** **5**
4.1 Tier 0: asymptotic defects . . . . . 5
4.2 Tier 1: hot path, low risk, mechanical . . . . . 6
4.3 Tier 2: structural changes, medium risk, large gains . . . . . 8
4.4 Tier 3: not recommended now . . . . . 9
4.5 Dismissed . . . . . 10

**5 Microbenchmarks of the patterns** **10**

**6 Invariants that must not move** **10**

**7 Proposed sequence** **11**

**8 Summary** **12**

**A Reference table of cited locations** **12**

# 1 Scope and method

**Object of study.** The `dev` branch of `F1R3FLY-io/f1r3node-rust` at commit `e8008c3` (merge of PR #781, 20 October 2026). The workspace has eleven crates; the audit covers `rholang`, `rho-pure-eval`, `rspace++`, `casper`, `block-storage`, `models`, `comm`, `shared` and `node`.

**Census.** A script counts the clone family of calls per crate, and separates production from test code. A file counts as test code when its path is under `tests/`, `benches/`, `scripts/`, `fuzz/`, or a test-utility module. Inline `#[cfg(test)]` items are removed by brace matching before the count.

**Deep reading.** The four highest-traffic subsystems — the Rholang reducer and substitution, the tuple space (RSpace), casper validation and merging, and the model, transport and storage layers — were read in full along their hot paths. Each finding records its location, the type that is copied, a cost class, whether it lies on a hot path, a proposed change, and a risk grade. Every line reference in this note was checked against the source at `e8008c3`.

**Cost classes.**

**Deep tree** a recursive structure (for example `Par`); one heap allocation per node.

**Large buffer** a `Vec<u8>` or `Vec<T>` whose size grows with the input.

**Small allocation** a 32-byte `Vec<u8>` hash, a short `String`.

**Pointer bump** `Arc`, `Bytes`, or an `imbl` persistent collection.

**Risk grades.** A change is **low** risk if it only changes ownership (move instead of copy, borrow instead of copy) and no observable output can change. It is **medium** risk if it changes a type or signature shared across crates, or touches code on which determinism depends without changing the computation. It is **high** risk if it touches hash bytes, serialisation formats, canonical sort order, or replay semantics.

**Limitations.** The node was not built or profiled for this note. The gains below are static estimates, supported by standalone microbenchmarks of the exact code patterns (Section 5). Section 7 proposes how to confirm them with the repository's own soak and latency tooling before and after each change.

# 2 Census

The densest files are `rholang/.../interpreter/reduce.rs` (230), `casper/.../engine/casper_launch.rs` (125), `node/.../runtime/setup.rs` (116), `rholang/.../rho_runtime.rs` (114) and `rspace++/.../replay_rspace.rs` (78). Two of these five (`casper_launch.rs` and `setup.rs`) run once at start-up and copy `Arc` handles; they are irrelevant to throughput. The count therefore tells where to look, not what to fix.

# 3 What a clone costs here

The cost of a clone depends on the type, and the types in this codebase fall into two clear groups.

| Crate | Prod. lines | `clone()` | `to_vec()` | `cloned()` | per kLoC |
|---|---|---|---|---|---|
| casper | 83,707 | 1,372 | 70 | 125 | 16.4 |
| rholang | 83,750 | 880 | 54 | 12 | 10.5 |
| rspace++ | 18,892 | 482 | 69 | 61 | 25.5 |
| node | 32,198 | 432 | 14 | 7 | 13.4 |
| models | 12,605 | 231 | 11 | 7 | 18.3 |
| block-storage | 9,745 | 184 | 38 | 36 | 18.9 |
| comm | 13,342 | 174 | 13 | 10 | 13.0 |
| shared, crypto, graphz, rho-pure-eval | 11,908 | 62 | 34 | 4 | 5.2 |
| **Total production** | **266,147** | **3,817** | **303** | **262** | **14.3** |
| Test code | | 7,045 | | | |

Table 1: Clone-family calls in production code at `e8008c3`.

**Already cheap.**

- `BlockHash`, `StateHash`, `Validator` and casper `ByteString` are `prost::bytes::Bytes`: `models/build.rs` lines 55–56 configure `.bytes(".casper")` and `.bytes(".routing")`. Most of the 1,372 casper clones are of these types.
- `KeyValueDagRepresentation` and `DagState` are built on `imbl` persistent maps; cloning them is $O(1)$.
- `DebruijnInterpreter` (`reduce.rs`:231–257) is about fourteen `Arc` fields; one clone is fourteen atomic increments and no heap work.
- The hot store's continuations are already `Vec<Arc<WaitingContinuation>>`. A comment at `hot_store.rs`:408–412 records that an earlier change removed a deep clone of persistent contract bodies on every produce. That change is the precedent this note extends.

**Expensive.**

- `Par` and every Rholang AST node. The `rhoapi` package has no `.bytes(...)` setting, and `Par` is nine `repeated` message vectors that recurse with no `Box`, `Arc` or `Rc`. Every clone is a full deep copy.
- `Env<Par>` (`rho-pure-eval/src/env.rs`:12) is a `HashMap<i32, Par>`. A clone deep-copies every bound process.
- `BlockMessage`. The hash fields are `Bytes`, but the body holds `Vec<ProcessedDeploy>`, each with a `String` term and a `Vec<Event>` deploy log. A clone costs on the order of the number of event-log entries.
- `DeployChainIndex` carries three `EventLogIndex` values of eleven hash sets each, plus a `StateChange` of three `DashMap`s.
- `Blake2b256Hash` is `pub struct Blake2b256Hash(pub Vec<u8>)` (`rspace++/../blake2b256_hash.rs`:17). It is not `Copy`; each clone is an allocation. It keys the RSpace event log and the merge channel maps.

**Diagnosis.** The repository was extracted from a Scala code base. In Scala, `env.put(x)` on an immutable map returns a new map that shares almost all of its structure with the old one, and

passing a value "by copy" is free. Several of the most expensive Rust sites reproduce that interface literally: they mutate in place *and* return a full clone, or they clone a collection, append one element, and store the result. `Env::put`, `SortedParMap::insert`, the stream reassembly buffer and `EventLogIndex::combine` all follow this pattern. The general remedy has two forms. Where the old value is dead after the call, move it. Where the old value is genuinely shared, use a persistent structure (`imbl` is already a workspace dependency) or `Arc`. The repository's own guideline in `CLAUDE.md` asks for "zero-cost abstractions, proper ownership"; the findings below are the places where the port has not yet met it.

# 4 Findings

The findings are in four tiers. Tier 0 holds asymptotic defects, where a fix changes the growth rate. Tier 1 holds hot-path, low-risk, mechanical changes. Tier 2 holds structural changes of medium risk with large gains. Tier 3 holds changes that are not recommended now.

## 4.1 Tier 0: asymptotic defects

**0.1 Stream reassembly is quadratic in message size.** `comm/src/rust/transport/stream_handler.rs`:482–491. For each incoming 256 KiB chunk, the whole buffer received so far is copied out of the cache, extended, and written back.

```
let received_bytes = content_data.to_vec();
let mut existing_data = cache.get(&streamed.key)
    .map(|entry| entry.value().clone()).unwrap_or_default();
existing_data.extend_from_slice(&received_bytes);
cache.insert(streamed.key.clone(), existing_data);
```

A 10 MiB block arrives in 40 chunks and causes about 200 MiB of copying; a 64 MiB stream causes about 8 GiB. `restore` (line 393) then clones the completed buffer again before it deletes the entry. *Fix:* `cache.entry(key).or_default().extend_from_slice(&content_data)`, and take the buffer at restore with `cache.remove`. Path: every received block and approved block. Risk: **low**; internal only, wire bytes unchanged.

**0.2 Replay scans its COMM list with a clone per step.** `rspace++/src/rspace/replay_rspace.rs`:1126–1153, `get_comm_or_candidate`. Each loop iteration runs `go(cs.clone())` on the remaining `Vec<COMM>`, and the miss branch returns `rem.to_vec()`. A scan over $n$ COMMs therefore copies $O(n^2)$ COMMs. In addition, `matches` and `was_repeated_enough_times` (lines 966, 973) take `COMM` and `Datum` by value, so callers clone both for every datum, every channel and every candidate COMM (lines 808, 959). *Fix:* iterate `comms.iter()` and return the first match; make both predicates take references. Visiting order is unchanged. Path: every replayed produce and consume, that is, every block validation. Risk: **low**.

**0.3 `Env::put` copies the whole environment on every binding.** `rho-pure-eval/src/env.rs`:31–40.

```
pub fn put(&mut self, a: A) -> Env<A> {
    Env { env_map: { self.env_map.insert(self.level, a);
                     self.env_map.clone() },
          level: self.level + 1, shift: self.shift }
}
```

Every caller folds `put` over a sequence, so binding $k$ values deep-copies $k(k+1)/2$ `Par`s. Callers include `build_env` on every COMM (`dispatch.rs`:17–19), `eval_new` (`reduce.rs`:1601–1607), match-case binding (`reduce.rs`:1424–1426) and guard evaluation (`matcher/match.rs`:96–98). `Env::shift` (line 48) clones the whole map as well, once per receive and once per substitution under a binder. The signature also has an aliasing hazard: it mutates `self` *and* returns a copy, so the caller's environment changes even if the caller uses only the return value. *Fix, first step:* add `put_owned(mut self, a) -> Self` and switch the fold sites, which own their accumulator. *Second step:* store the map as `Arc<HashMap>` with `Arc::make_mut`, or as an `imbl::HashMap`; then `shift` and every per-branch `env.clone()` become pointer bumps (see 2.1). Risk: **low** for the first step (behaviour identical at the fold sites), **medium** for the second (type crosses crates).

**0.4 `EventLogIndex::combine` in folds is quadratic in chain length.** `rspace++/.../merger/event_log_index.rs`:443ff. `combine(x: &Self, y: &Self)` builds the union of all eleven sets with `.union(..).cloned().collect()`. Casper folds with it (`deploy_chain_index.rs`:87, 91; `dag_merger.rs`:1408–1435), so an accumulator over $n$ deploys is re-copied $n$ times. *Fix:* add `combine_into(&mut self, &other)` that extends in place; keep `combine` for other callers. Set semantics do not depend on insertion order. Risk: **low**–**medium**.

**0.5 `eval_match` copies the remaining cases on every failed case.** `reduce.rs`:1513 passes `mat.cases.clone()`; each failed case then does `state = (_target, case_rem.to_vec())` (lines 1453, 1479). A match with $n$ cases that falls through to the last one copies $O(n^2)$ case bodies. *Fix:* loop over an index into `&mat.cases`. Risk: **low**.

**0.6 `SortedParMap::extend` re-sorts the whole map once per element.** `models/src/rust/sorted_par_map.rs`:43–53. `insert` clones the map and rebuilds the sorted form; `extend` calls `insert` in a loop, discarding each result, then rebuilds once more. `SortedParHashSet` has the same shape. The reducer uses these for the Rholang `++`, `+`, `-` operations on maps and sets. *Fix:* insert all pairs, then build the sorted form once. Risk: **medium**; this is canonical-form code, and the fix needs a differential test that compares sorted outputs (Section 6).

**0.7 Light-finalised-state sync state is copied per block.** `casper/.../engine/lfs_block_requester.rs`:136–148, 339–360. The requester's `HashMap` and `HashSet` state is cloned on every `add` and `done`, so sync of $n$ blocks costs $O(n^2)$. This affects only initial sync, so it is recorded here for completeness and scheduled last. Risk: **low**.

## 4.2 Tier 1: hot path, low risk, mechanical

Each item below changes ownership only. None changes a value that is hashed, charged, sorted or logged.

| # | Location | Change |
|---|---|---|
| 1.1 | `reduce.rs`:401–408 (`eval_inner` fan-out) | `terms` is owned and dead after the map; use `into_iter()` instead of `term.clone()`. Make `split` take `&Blake2b512Random`: the split is *required*, the copy of the parent generator is not. Applies to every `Par` with two or more terms. |
| 1.2 | `substitute.rs`:205 | `expr.clone().expr_instance` deep-copies every `Expr` of every substituted `Par` to inspect its variant. Use `match &expr.expr_instance`. Same pattern at lines 165, 184, 407, 630, 975. |
| 1.3 | `substitute.rs`:63, 90 | `substitute_and_charge` clones its input only so that the error path can read `encoded_len()`. Add a by-value variant; compute the length before the move. Charge values and order unchanged. |
| 1.4 | `reduce.rs`:1725 | `eval_expr_to_par` deep-copies each expression to match on it. Match by reference. |
| 1.5 | `reduce.rs`:1740, 7485 | `method_table()` builds a `HashMap` of about 60 `String` keys and boxed closures on *every* method call. Replace with a `match` on the name (an allocation fix rather than a clone fix). |
| 1.6 | `reduce.rs`:2038–2916 (32 sites) | `&p1.clone().unwrap()` deep-copies an operand to borrow it. Use `p1.as_ref().unwrap()`. Similar at 1283, 1442, 1694, 2679. |
| 1.7 | `reduce.rs`:563, 648–652 | `produce_inner` and `consume_inner` clone channel, data, sources, patterns, the whole continuation body and the guard; the originals are used again only in the `persistent` branch. Clone only when persistent. |
| 1.8 | `charging_rspace.rs`:89, 113 | The full RSpace result (continuation body and every matched datum) is cloned so that `handle_result` can read it. Pass by reference. |
| 1.9 | `dispatch.rs`:53 | `build_env(data_list.clone())`; build the random-state vector first, then move. The *required* random-state merge order is unchanged. |
| 1.10 | `space_matcher.rs`:94 | `rollback.push((channel.clone(), original.clone()))` copies the whole indexed data vector per matched channel. Use `std::mem::replace`. |
| 1.11 | `space_matcher.rs`:136–140 | Channel and pattern `Par`s are cloned into pairs for every candidate continuation probed. Pair references. |
| 1.12 | `hot_store.rs`:414, 471–474 | Installed (system) continuations are stored by value and deep-cloned into a fresh `Arc` on every produce on an installed channel. Store them as `Arc`, as the ordinary continuations already are. |

| # | Location | Change |
|---|---|---|
| 1.13 | shared/.../key_value_store.rs:49–52 | get_one clones the first value instead of moving it: a second full copy of every value read, including every stored block and every trie node. Use values.into_iter().next().flatten(). Same at key_value_typed_store_impl.rs:76. |
| 1.14 | interpreter_util.rs:518–519, 609–610; proto_util.rs:339 | Replay clones the full deploy and system-deploy lists once to extract them and again per replay attempt. The consumer only iterates. Take &[ProcessedDeploy]. |
| 1.15 | interpreter_util.rs:386; block_creator.rs:3482 | parents.clone() of Vec<BlockMessage> into compute_parents_post_state; at 386 the original is dead, and in the proposer it is copied on every retry. Pass a slice. |
| 1.16 | block_processor.rs:542 | block.to_proto().encode_to_vec().len() clones the body twice and encodes it only to record a size metric. Use encoded_len(), as block_creator.rs:3648 already does. |
| 1.17 | block_processor.rs:470, 499, 1097; casper_launch.rs:302, 322 | Whole blocks cloned into PrettyPrinter::build_string for logging. Use the borrowing build_string_block_message(&block, ...). No risk. |
| 1.18 | conflict_set_merger.rs:1194 | fold_rejection clones the balance map for every branch. Take &HashMap and return a new map only on success. Fold order unchanged. |
| 1.19 | replay_rspace.rs:649–653 | locked_consume deep-copies all data on the channels only to collect produce hashes, then fetches the same data again. Reuse it. |

### 4.3 Tier 2: structural changes, medium risk, large gains

**2.1 Persistent `Env`.** After 0.3, every spawned branch in eval_inner, every shift, and every substitution under a binder still deep-copies the environment. An imbl::HashMap or Arc-backed map turns all of these into pointer bumps. This is the largest single structural win in the interpreter. The type is public in rho-pure-eval, so the change crosses crates.

**2.2 `Arc<Datum>` in the hot store.** get_data (hot_store.rs:631–641) returns the whole Vec<Datum<A>> for a channel on every produce and consume, because ShardedMap::get returns .cloned() (lines 107–112). On a busy channel this is the largest per-operation copy in RSpace. Add get_data_arc returning Vec<Arc<Datum<A>>>, mirroring the existing get_continuations_arc; keep get_data for other callers. There is a second, probable effect: after each soft checkpoint the imbl shards are shared with the snapshot, so the next get_mut copies a map node, which copies every sibling value in it. With Arc values those copies become pointer bumps. This second effect should be confirmed against the imbl source before it is claimed.

**2.3 `Arc<DeployChainIndex>` and `Arc<BlockIndex>`.** The block-index cache returns deep copies (`interpreter_util.rs`:1317, `(*cached.value()).clone()`), so every merge deep-copies the index of every block in scope, event logs included. The merger then clones chains again (`dag_merger.rs`:433, 459, 468, 734–737). `DeployChainIndex` has hand-written `Eq`, `Hash` and `Ord` over `deploys_with_cost` only, so `Arc` wrapping keeps merge order identical by construction. The one mutation, `stamp_prior_rejections` (`dag_merger.rs`:602–604), becomes `Arc::make_mut`.

**2.4 Borrowing merge accessors.** `dag_merger.rs`:1504–1507 defines `|chain| Ok(chain.state_changes.clone())` and a similar accessor for mergeable channels. The merger calls them repeatedly per item, branch and rejection option (`conflict_set_merger.rs`:217, 553, 908, 1105). Returning references needs a `combine_ref` on `StateChange` and a change to the generic `merge` signature. `dag_merger.rs`:212–213 also clones two growing datum maps per chain; an undo log would remove that.

**2.5 Encode once, send many.** `stream_mult` (`grpc_transport_client.rs`:590) streams to each peer separately. Each stream encodes the packet into the cache, decodes it, clones it, LZ4-compresses the whole block and copies each chunk (`stream_observable.rs`:147, `chunker.rs`:22–24). Compress and chunk once, and share the chunks as `Bytes` slices. Wire bytes are identical.

**2.6 Borrowing protobuf conversion.** `BlockMessage::to_proto(&self)` and `Body::to_proto` clone every deploy and justification before converting (`casper_message.rs`:834, 956–968, 1508–1514). This runs at store, gossip, hashing and the size metric. Add `to_proto_ref` methods that map over references. Output bytes are identical, but because `hash_block` uses this path, the change needs a golden-hash test.

**2.7 DAG representation without LMDB scans.** Not a clone, but the same kind of waste: `get_representation_internal` (`block_dag_key_value_storage.rs`:1199–1207) rebuilds `latest_messages` and `invalid_blocks` by scanning and decoding LMDB on every call, and `admit_dag_contains` builds a whole representation to answer one membership query. A `contains` method that reads `dag_state` is **low** risk; caching the two collections in `DagState` is **medium** (restart recovery must rebuild them).

**2.8 Sorter ownership.** The sort matchers clone whole scored vectors to separate terms from scores (for example `send_sort_matcher.rs`:26, `sorted_data.clone().into_iter().map(|p| p.term)`), and several clone an unsorted node only to overwrite the field they just copied. Each recursion level copies everything below it, which is quadratic in depth. The unzip helper `split_scored_terms` already exists in `par_sort_matcher.rs`. Comparisons and order are untouched, but the code is the canonical form of every term, so this is graded **medium** and placed behind the differential test.

### 4.4 Tier 3: not recommended now

- **`Blake2b256Hash` as `[u8;32]`.** Removes an allocation per clone across RSpace, but its derived serde writes a length-prefixed vector. Cold-store keys, history-trie keys and event-log hashing would change unless a byte-identical custom serialiser is written. **high**.

- **`Bytes` or `Arc` inside `rhoapi`.** Would make `Par` clones cheap at the root, but touches the serde and schema derives and the `locally_free` handling in `models/build.rs`. **high**.
- **Removing the second sort of maps and sets.** Depends on sort idempotence and on identical deduplication semantics. **high** without a proof.
- **`continuation_identity`** (`hot_store.rs`:1012). It formats a `{:?}` string of every continuation on a channel at each `put_continuation`. Not a clone, but expensive; it decides store contents and therefore the state hash, so a replacement needs an equivalence argument. **medium**–**high**.
- **Zero-copy LMDB reads.** Would change the on-disk format. **high**.

### 4.5 Dismissed

Start-up wiring in `node/.../setup.rs`, `node_runtime.rs` and `casper_launch.rs`; `Arc` handle bumps for spawned tasks; clones of `BlockHash`, `StateHash` and `Validator`; per-request clones in `web_api.rs` (small strings and hashes); and clique oracle map clones (small validator sets). These account for well over a third of the census and for a negligible part of the cost.

## 5 Microbenchmarks of the patterns

Four patterns were reproduced in a standalone Rust program with the same shape as the repository code, and timed against the proposed fix (release build, stable toolchain, single thread). A synthetic `Tree` with nested `Vec`s stands in for `Par`. The numbers measure the pattern, not the node; they show growth rates, not production latency.

| Pattern | Size | Current | Fixed | Ratio |
|---|---|---|---|---|
| Stream reassembly (0.1) | 1 MiB | 1.63 ms | 0.035 ms | 46× |
| | 10 MiB | 84.5 ms | 0.82 ms | 103× |
| | 64 MiB | 5,562 ms | 43.9 ms | 127× |
| `Env::put` fold (0.3), 85-node terms | 2 bindings | 14.0 µs | 5.1 µs | 2.7× |
| | 8 bindings | 126.7 µs | 23.1 µs | 5.5× |
| | 32 bindings | 2,160 µs | 128 µs | 16.9× |
| Replay COMM scan (0.2), match last | 8 COMMs | 5.3 µs | 0.6 µs | 9.5× |
| | 64 COMMs | 542 µs | 8.1 µs | 67× |
| | 256 COMMs | 7,831 µs | 28.7 µs | 273× |
| Fan-out, 16 subterms (1.1) | 121-node terms | 134.7 µs | 62.6 µs | 2.2× |

Table 3: Pattern microbenchmarks. “Fixed” is the change proposed in this note.

The first three rows grow with input size, as the analysis predicts; the last is a constant factor, as expected for a single redundant copy. The stream result deserves emphasis: at 64 MiB the current pattern spends more than five seconds copying buffers that the fixed pattern appends in place.

## 6 Invariants that must not move

Some clones in the hot paths are *required*, and the plan must leave them alone.

1. **Random-number splitting.** Each parallel branch receives a generator split from its parent by index (`reduce.rs`:408). The split determines every unforgeable name and must stay. Only the extra copy of the parent before splitting goes.
2. **Cost accounting.** The amount and order of every charge must stay identical. Items 1.3, 1.8 and the cost helpers that take a whole value to read its length change only how a length is obtained.
3. **Canonical form.** Sorting defines term equality and hashes. Changes to the sorter (0.6, 2.8) must produce byte-identical output.
4. **Hash and serialisation bytes.** `hash_block`, event-log hashes, trie keys and LMDB values must not change.
5. **Replay semantics.** COMM matching order in replay must be preserved; 0.2 keeps the iteration order.

**Guards.** Three tests make the medium-risk work safe.

- *Golden state hashes.* Run a fixed corpus of Rholang deploys (the existing test contracts, `PoS.rhox`, and the registry) from genesis and record the post-state hash and event-log hash of each. Every pull request in Tiers 1 and 2 must reproduce them exactly.
- *Proto round trip.* For a corpus of blocks, assert that new and old `to_proto` produce identical encoded bytes and identical block hashes.
- *Sorter differential.* For random `Par`s (the `fuzz/` crate already has generators), assert that old and new sort matchers produce identical terms and score trees.

## 7 Proposed sequence

Each step is one small pull request to `dev`, with the `ci-heavy` label, so the integration pipeline runs before the merge queue.

1. **Guards first.** Land the golden-hash and proto round-trip tests, and record a baseline with `scripts/bench/casper-soak.sh` and `latency-benchmark.sh`: block validation time, replay time, propose time, peak resident memory, and the `hot-store.get_data.calls` and `hot-store.get_data.history_fill` metrics.
2. **Asymptotic, low risk:** 0.1 (stream), 0.2 (replay scan), 0.3 first step (`put_owned`), 0.5 (match cases). Four independent PRs.
3. **Interpreter mechanical:** 1.1–1.9. Mostly within `reduce.rs`, `substitute.rs`, `dispatch.rs`, `charging_rspace.rs`.
4. **RSpace and storage mechanical:** 1.10–1.13, 1.19.
5. **Casper mechanical:** 1.14–1.18. Together with 0.1, these remove about six of the roughly eight full body copies an incoming block undergoes between receipt and storage.
6. **Structural:** 0.4, 2.1, 2.2, 2.3, 2.4 — one at a time, each with a fresh soak comparison.
7. **Transport and model:** 2.5, 2.6, 2.7, then 0.6 and 2.8 behind the sorter differential.

8. **Prevention.** Add an advisory CI job that runs Clippy with `-W clippy::redundant_clone` and `-W clippy::needless_pass_by_value` on the interpreter, RSpace and merge modules. The current `cargo clippy` step uses default lints, and `redundant_clone` is a nursery lint, so it does not run today. Keep the job non-blocking at first; the pedantic lint produces noise on cold code.

**Expected effect.** Without profiles, an honest estimate is qualitative. Steps 2–5 are mechanical and should remove most of the avoidable deep copies on the evaluation, replay and block-intake paths. The largest wall-clock gains are expected where growth rates change: large block transfer (0.1), replay of blocks with many COMMs on a channel (0.2), contracts that bind many names or match on many cases (0.3, 0.5), and merges over long deploy chains (0.4, 2.3). A soak comparison after step 5 will show whether Tier 2 is needed for current workloads or can wait.

## 8 Summary

The clone count is high, but most clones are cheap: the consensus hash types are already reference-counted, and the DAG representation is already persistent. The cost is concentrated where the Scala port copies deep values to imitate immutable-collection semantics. Seven sites are asymptotic defects, and four of them are low-risk, few-line fixes. About twenty further hot-path sites need only a change of ownership. The structural changes — a persistent `Env`, `Arc`-shared data in the hot store and in the merge indices — extend a pattern the code base has already applied successfully to continuations. All of this can proceed without touching random-number splitting, cost order, canonical sorting, hash bytes or replay order, provided the golden-hash, round-trip and sorter-differential guards land first.

## A Reference table of cited locations

| Item | File : line | Path | Risk |
|---|---|---|---|
| 0.1 | `comm/src/rust/transport/stream_handler.rs`:482–491, 393 | per block | **low** |
| 0.2 | `rspace++/src/rspace/replay_rspace.rs`:1126–1153, 966, 973 | per replay op | **low** |
| 0.3 | `rho-pure-eval/src/env.rs`:31–51 | per COMM | **low/medium** |
| 0.4 | `rspace++/src/rspace/merger/event_log_index.rs`:443ff | per merge | **low/medium** |
| 0.5 | `rholang/src/rust/interpreter/reduce.rs`:1453, 1479, 1513 | per match | **low** |
| 0.6 | `models/src/rust/sorted_par_map.rs`:43–69 | map/set ops | **medium** |
| 0.7 | `casper/src/rust/engine/lfs_block_requester.rs`:136–148 | sync only | **low** |
| 1.1–1.9 | `reduce.rs`, `substitute.rs`, `dispatch.rs`, `charging_rspace.rs` | per eval / COMM | **low** |
| 1.10–1.13 | `space_matcher.rs`, `hot_store.rs`, `key_value_store.rs` | per op / read | **low** |
| 1.14–1.18 | `interpreter_util.rs`, `block_creator.rs`, `block_processor.rs`, `conflict_set_merger.rs` | per block | **low** |
| 2.1 | `rho-pure-eval/src/env.rs` | per eval | **medium** |

| Item | File : line | Path | Risk |
|---|---|---|---|
| 2.2 | `rspace++/src/rspace/hot_store.rs`:107–112, 631–641 | per op | **medium** |
| 2.3 | `interpreter_util.rs`:1317; `dag_merger.rs`:433–737 | per merge | **medium** |
| 2.4 | `dag_merger.rs`:212, 1504–1507 | per merge | **medium** |
| 2.5 | `grpc_transport_client.rs`:590; `chunker.rs`:22 | per broadcast | **low/medium** |
| 2.6 | `models/src/rust/casper/protocol/casper_message.rs`:834–1514 | per block | **medium** |
| 2.7 | `block_dag_key_value_storage.rs`:1199–1207 | per block | **low/medium** |
| 2.8 | `models/src/rust/rholang/sorter/*` | per substitution | **medium** |